

ARTIFICIAL INTELLIGENCE IN FINTECH / WEEK 07

Evaluating a Fixed Backtest

Keep the Week 6 strategy and 4 bp cost assumption fixed. Measure return, risk and trading activity; then examine uncertainty and the consequences of selecting a strategy after seeing results. Do not change portfolio weights while comparing metrics.

Issue	Question	Page
1	Which daily returns are being evaluated?	3
2	Which average describes the account's growth?	4
3	What does annualizing half a year actually mean?	5
4	When does square-root scaling describe volatility?	6
5	Which daily reference return belongs in the calculation?	7
6	What does the Sharpe ratio divide by?	8
7	Which observations enter downside deviation?	9
8	How does the Sortino ratio change the question?	10
9	How far is wealth below its previous peak?	11
10	How long does a drawdown last?	12
11	What must match in a benchmark comparison?	13
12	Does the result depend on the chosen period?	14
13	Why might adjacent returns not be independent?	15
14	What does a block-bootstrap interval describe?	16
15	How much does the block length matter?	17
16	What changes when we select the best of several rules?	18
17	What happens to the selected rule in the later period?	19
18	Which conclusion is supported by the complete evidence?	20

Use the examples in order. Each issue ends with two discussion questions, not a separate submission. Market observations and teaching assumptions are identified separately.

SHARED DATA AND SETTINGS

Prepare the common example

AAPL, January 2-June 28, 2024, provider Close in USD/share. Positive five-observation return gives stock exposure; otherwise cash. Decision after Close t , execution at Close $t+1$, first earned return ending $t+2$. Start in cash with USD 1,000; no dividends, taxes, interest or final new trade. Costs use Week 6's linear opening-wealth approximation.

Before running: Should the final balance match Week 6 rather than change when a new metric is introduced?

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import yfinance as yf
aapl = yf.Ticker("AAPL").history(start="2024-01-02",
    end="2024-07-01", auto_adjust=False, actions=True)
aapl.index = aapl.index.strftime("%Y-%m-%d")
close = aapl["Close"]
assert close.index.is_unique and close.index.is_monotonic_increasing
assert np.isfinite(close).all() and (close > 0).all()
signal = (close / close.shift(5) - 1).gt(0).astype(float)
position = signal.shift(2).fillna(0)
after_close = signal.shift(1).fillna(0)
after_close.iloc[-1] = position.iloc[-1]
turnover = (after_close - position).abs()
asset_return = close.pct_change(fill_method=None)
assert asset_return.iloc[1:].notna().all()
gross = position * asset_return.fillna(0)
net = gross - 0.0004 * turnover
wealth = 1000 * (1 + net).cumprod()
r = net.iloc[1:]
pd.DataFrame({"Last date": [close.index[-1]], "Daily intervals": [len(r)],
    "Net wealth (USD)": [round(wealth.iloc[-1], 2)]})
```

Last date	Daily intervals	Net wealth (USD)
2024-06-28	123	1056.07

The displayed balance must match Week 6: USD 1,056.07, allowing display rounding. January 2 is the starting boundary, not an extra daily return. The first five unavailable feature values produce cash signals. All main statistics use the same 123 daily net returns.

ISSUE 01

Which daily returns are being evaluated?

Count return intervals, not prices or orders. The sample contains 124 closing prices and 123 close-to-close intervals; a cash day is still an evaluated interval.

The symbol $r_{p,t}^{\text{net}}$ denotes the net portfolio return ending on date t ; p identifies the portfolio. In the code, r stores these returns after excluding the initial boundary. The initial USD 1,000 remains part of wealth and drawdown.

Before running: Does a negative stock return necessarily imply a negative portfolio return while the account is in cash?

```
shown = pd.DataFrame({"Stock (%)": 100 * asset_return,
                      "Gross (%)": 100 * gross, "Net (%)": 100 * net})
shown = shown.loc[["2024-01-03", "2024-01-11", "2024-01-12"]].round(4)
shown.insert(0, "Return ends", shown.index)
shown
```

Return ends	Stock (%)	Gross (%)	Net (%)
2024-01-03	-0.7488	-0.0	-0.0
2024-01-11	-0.3223	-0.0	-0.04
2024-01-12	0.1778	0.1778	0.1778

January 3 has zero portfolio return while cash is held. January 11 has -0.04% net because the closing purchase pays a fee. January 12 earns the stock return. Deleting cash dates would change the sample and exaggerate the frequency of exposure.

Discuss

1. Why should January 2 not be counted as a 124th daily return? Distinguish the starting balance from a holding interval.
2. If returns are missing while stock is held, why must the evaluation stop rather than silently dropping those dates?

Sources: [1], [5]. See the references page.

ISSUE 02

Which average describes the account's growth?

The arithmetic mean describes the average daily observation. The geometric mean is the constant daily rate that reproduces the observed compound growth. Use each for the question it answers.

There are $n = 123$ daily net portfolio returns. Their arithmetic mean is the sum divided by n . W_0 is initial wealth and W_n is final wealth, both in USD. Let g be the constant daily growth rate; its n return factors must reproduce final wealth divided by initial wealth.

$$\bar{r}_p^{\text{net}} = \frac{1}{n} \sum_t r_{p,t}^{\text{net}}, \quad (1 + g)^n = \frac{W_n}{W_0}$$

$$\bar{r}_p^{\text{net}} \approx \frac{0.064470}{123} \approx 0.000524$$

$$g \approx (1056.07442/1000)^{1/123} - 1 \approx 0.000444$$

Before running: Which daily average reproduces the final USD 1,056.07 when repeated 123 times?

```
n = len(r)
mean_return = r.mean()
geometric = np.exp(1/np.log(1+r).mean())
pd.DataFrame({"Days": [n], "Arithmetic (%)": [100 * mean_return],
              "Geometric (%)": [100 * geometric],
              "Total (%)": [100 * (wealth.iloc[-1] / 1000 - 1)]}).round(4)
```

Days	Arithmetic (%)	Geometric (%)	Total (%)
123	0.0524	0.0444	5.6074

The arithmetic mean is 0.0524% per day; the geometric mean is 0.0444%. The observed total gain is 5.6074%, not 123 times either average. These descriptive values are not forecasts.

Discuss

1. Which mean belongs in the Sharpe numerator, and why would substituting compounded growth answer a different question?
2. Can an arithmetic mean alone reconstruct the account balance without the observed returns? Explain what information about compounding is missing.

Sources: [2], [5]. See the references page.

ISSUE 03

What does annualizing half a year actually mean?

Annualizing a partial-year gain asks what constant annual rate would reproduce that growth under a stated time convention. It does not create a year of observed evidence.

Use 252 trading intervals per year and $n = 123$ observed intervals. The annualized rate R satisfies the first equation; raising both sides to $252/n$ gives the second. Wealth must remain positive.

$$(1 + R)^{n/252} = \frac{W_n}{W_0} \implies R = \left(\frac{W_n}{W_0}\right)^{252/n} - 1$$

$$R \approx (1056.07442/1000)^{252/123} - 1 \approx 0.118265$$

Before running: Will extending this positive partial-year rate to 252 intervals produce a larger number than the observed gain?

```
annualized_return = (wealth.iloc[-1] / 1000) ** (252 / n) - 1
pd.DataFrame({"Observed days": [n],
              "Observed gain (%)": [100 * (wealth.iloc[-1] / 1000 - 1)],
              "Annualized (%)": [100 * annualized_return]}).round(3)
```

Observed days	Observed gain (%)	Annualized (%)
123	5.607	11.827

The observed gain is 5.607%; the annualized teaching calculation is 11.827%. It assumes the stated trading-interval scale, not exact elapsed calendar years or persistence of the return process.

Do not present the annualized figure as a realized annual result. GIPS reporting rules do not permit annualizing periods shorter than one year; this classroom calculation is not a compliant performance presentation.

Discuss

1. Which number should describe what was actually earned from January 2 to June 28? Give the period and rate.
2. Would inserting 365 while retaining 123 trading intervals be consistent? Explain what time definition would first need to change.

Sources: [5]. See the references page.

ISSUE 04

When does square-root scaling describe volatility?

Volatility measures dispersion, not the size of the final loss. The usual annualized figure multiplies daily standard deviation by the square root of 252.

Let s be sample standard deviation of the 123 daily net returns, using divisor $n-1$. The square-root rule follows for sums of equal-variance, uncorrelated returns because their variances add. It is not an exact identity for compounded annual simple returns.

$$s = \sqrt{\frac{\sum_t (r_{p,t}^{\text{net}} - \bar{r}_p^{\text{net}})^2}{n-1}}, \quad \hat{\sigma}_{\text{annual}} = s\sqrt{252}$$

$$s \approx \sqrt{0.0201590/122} \approx 0.012854, \quad 0.012854\sqrt{252} \approx 0.204059$$

Before running: Is annualized volatility 252 times daily volatility, or about 15.87 times?

```
daily_sd = r.std(ddof=1)
annual_sd = daily_sd * np.sqrt(252)
pd.DataFrame({"Days": [len(r)], "Daily volatility (%)": [100 * daily_sd],
              "Annualized (%)": [100 * annual_sd]}).round(3)
```

Days	Daily volatility (%)	Annualized (%)
123	1.285	20.406

Daily volatility is 1.285%; the conventionally scaled value is 20.406%. It does not mean a 20.406% maximum loss or a guaranteed range. Dependence and changing variance require further checks.

Discuss

1. Why do variances, rather than standard deviations, add under the stated uncorrelated-sum assumption?
2. If adjacent returns have positive covariance, which term is omitted by multiplying variance by 252 alone?

Sources: [2], [3], [5]. See the references page.

ISSUE 05

Which daily reference return belongs in the calculation?

A daily strategy return must be compared with a daily reference. Subtracting an annual percentage directly from every daily return changes the financial meaning.

The main Sharpe calculation uses an assumed constant zero cash/reference return. Sortino also uses a zero minimum acceptable return. These are teaching choices, not observed Treasury rates. To illustrate a different target only, convert a hypothetical 5% effective annual rate to a constant daily rate q .

$$(1 + q)^{252} = 1.05 \implies q = 1.05^{1/252} - 1$$

$$q \approx 0.00019363 \approx 1.9363 \text{ bp/day}$$

Before running: Will compounding the converted daily target recover the specified annual target?

```
annual_targets = np.array([0.0, 0.05])
daily_targets = (1 + annual_targets) ** (1 / 252) - 1
pd.DataFrame({"Assumed annual (%)": 100 * annual_targets,
             "Daily target (bp)": np.round(10000 * daily_targets, 4),
             "Compounded annual (%)": np.round(100 * ((1 + daily_targets) ** 252 - 1), 4)})
```

Assumed annual (%)	Daily target (bp)	Compounded annual (%)
0.0	0.0	0.0
5.0	1.9363	5.0

The hypothetical 5% target becomes 1.9363 bp per day. The main examples below retain 0%, so the strategy itself and its wealth do not change when we discuss the conversion.

Discuss

1. What sources and date alignment would be required to replace the zero assumption with an observed cash-return series?
2. Does increasing a minimum acceptable return change the account's historical wealth, or only its assessment relative to that target?

Sources: [2], [4]. See the references page.

ISSUE 06

What does the Sharpe ratio divide by?

The Sharpe ratio compares mean differential return with the standard deviation of those same differential returns. Its denominator is not downside deviation or maximum drawdown.

Let q be the constant daily cash/reference return, here zero. Subtract q from each net return. Because q is constant, the standard deviation stays s . The ratio is dimensionless; the annualized figure uses the square-root convention and its limitations.

$$\widehat{SR}_{\text{annual}} = \frac{\bar{r}_p^{\text{net}} - q}{s} \sqrt{252}, \quad s > 0$$

$$\widehat{SR} \approx \frac{0.00052415 - 0}{0.01285448} \sqrt{252} \approx 0.6473$$

Before running: Should the sign of the ratio match the sign of the mean differential return when the denominator is positive?

```
reference_return = 0.0
differential = r - reference_return
assert differential.std(ddof=1) > 0
sharpe_daily = differential.mean() / differential.std(ddof=1)
sharpe_annual = sharpe_daily * np.sqrt(252)
pd.DataFrame({"Reference (%/day)": [0.0],
              "Daily Sharpe": [sharpe_daily], "Annualized Sharpe": [sharpe_annual]}).round(4)
```

Reference (%/day)	Daily Sharpe	Annualized Sharpe
0.0	0.0408	0.6473

The annualized estimate is 0.6473 for these 123 net returns. It is not a significance test. A zero-return cash series has zero denominator, so its Sharpe ratio is undefined—not automatically zero.

Discuss

1. If the reference return varies by date, why must the denominator use the matched differential-return series instead of the strategy's standard deviation alone?
2. Can an estimated ratio from one short sample establish future superiority? Identify what uncertainty this single value omits.

Sources: [2], [3]. See the references page.

ISSUE 07

Which observations enter downside deviation?

Downside deviation counts shortfalls below a specified target. This lesson divides by all 123 observations, including zero contributions from dates that meet the target.

q is the daily minimum acceptable return, here zero. The signed shortfall d is the smaller of net return minus q and zero. DD is the root mean squared shortfall, in decimal-return units. It is not the standard deviation of negative returns around their own mean.

$$d_t = \min(r_{p,t}^{\text{net}} - q, 0), \quad \text{DD}_q = \sqrt{\frac{1}{n} \sum_t d_t^2}$$

$$\text{DD}_0 \approx \sqrt{0.0060338/123} \approx 0.007004$$

Before running: Does January 12 contribute its positive return squared, or zero, to the downside calculation?

```
shortfall = np.minimum(r - 0.0, 0)
downside = np.sqrt(np.mean(shortfall ** 2))
shown = pd.DataFrame({"Net (%)": 100 * r,
    "Shortfall (pp)": 100 * shortfall,
    "Squared (pp²)": (100 * shortfall) ** 2})
shown = shown.loc[["2024-01-11", "2024-01-12", "2024-01-17"]].round(5)
shown.insert(0, "Date", shown.index)
shown
```

Date	Net (%)	Shortfall (pp)	Squared (pp²)
2024-01-11	-0.04	-0.04	0.0016
2024-01-12	0.17781	0.0	0.0
2024-01-17	-0.55735	-0.55735	0.31064

January 12 contributes zero. Across all 123 days, 47 fall below zero and downside deviation is 0.7004 percentage points per day. Dividing by only 47 would give a different, larger root mean square.

Discuss

1. Why must the denominator convention accompany a reported Sortino ratio? Compare all-date and below-target-only versions.
2. If the target becomes positive, could a positive daily return contribute to downside deviation? Specify the condition.

Sources: [4]. See the references page.

ISSUE 08

How does the Sortino ratio change the question?

Sortino uses target shortfalls in the denominator; Sharpe uses total differential-return variability. Neither is a complete description of risk.

Use the same mean daily net return and zero target q . DD uses all dates as specified in Issue 7. The annualized Sortino below multiplies the daily ratio by $\sqrt{252}$ as a reporting convention, not an identity for annual downside risk.

$$\widehat{\text{Sortino}}_{\text{annual}} = \frac{\bar{r}_p^{\text{net}} - q}{\text{DD}_q} \sqrt{252}, \quad \text{DD}_q > 0$$

$$\widehat{\text{Sortino}} \approx \frac{0.00052415 - 0}{0.00700393} \sqrt{252} \approx 1.1880$$

Before running: Will changing the denominator change the historical account balance?

```
assert downside > 0
sortino_daily = (r.mean() - 0.0) / downside
sortino_annual = sortino_daily * np.sqrt(252)
pd.DataFrame({"Ratio": ["Sharpe", "Sortino"],
              "Daily": [sharpe_daily, sortino_daily],
              "Annualized": [sharpe_annual, sortino_annual]}).round(4)
```

Ratio	Daily	Annualized
Sharpe	0.0408	0.6473
Sortino	0.0748	1.188

Sortino is 1.1880 and Sharpe is 0.6473 on their annualized scales. Both describe the same USD 1056.07 ending balance. A different ratio is not a different trading result.

If DD is zero, Sortino is undefined. When mean excess return is negative, a larger denominator makes the ratio less negative; that alone does not mean downside outcomes improved.

Discuss

1. Would a zero-target Sortino and a positive-target Sortino isolate differences between strategies? State what needs to be held fixed.
2. Can a high Sortino value exclude a long drawdown? Explain which information about the order of returns is absent from this ratio.

Sources: [2], [4]. See the references page.

ISSUE 09

How far is wealth below its previous peak?

Drawdown asks how far current wealth has fallen below its previous highest value. It depends on the path, not just the final return.

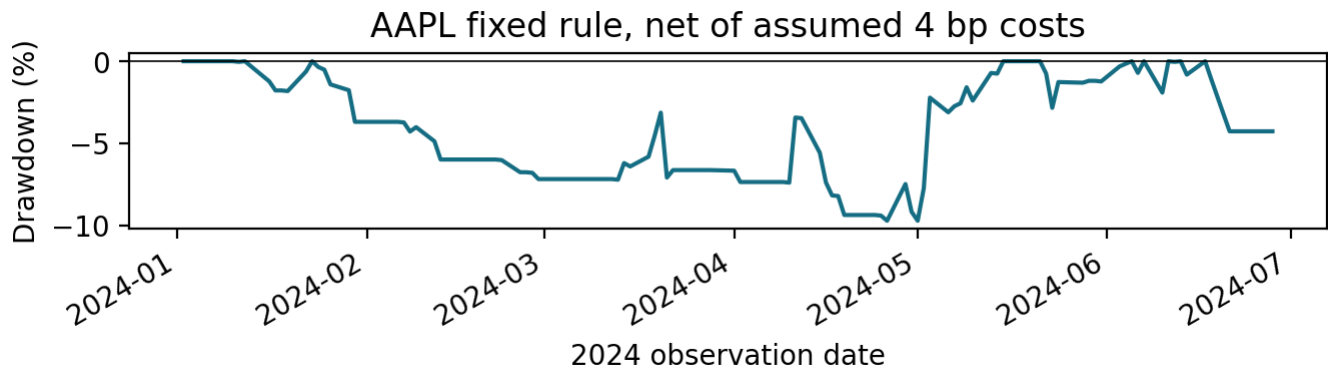
W is net wealth in USD. M is the running peak, including the initial USD 1,000. D is the fractional drawdown: zero at a peak and negative below it. We report maximum drawdown as a positive loss magnitude, $-\min(D)$.

$$M_t = \max(W_0, \dots, W_t), \quad D_t = \frac{W_t}{M_t} - 1$$

$$D_{\text{trough}} \approx \frac{904.24384}{1001.71629} - 1 \approx -0.097305$$

Before running: Must drawdown return to zero merely because the account gains on one day?

```
peak = wealth.cummax().clip(lower=1000)
drawdown = wealth / peak - 1
max_drawdown = -drawdown.min()
ax = (100 * drawdown).set_axis(pd.to_datetime(drawdown.index)).plot(
    figsize=(7, 2.1), color="#156d83")
ax.axhline(0, color="black", linewidth=0.6)
ax.set(title="AAPL fixed rule, net of assumed 4 bp costs",
       xlabel="2024 observation date", ylabel="Drawdown (%)")
plt.tight_layout()
plt.gcf()
```



The deepest drawdown occurs on 2024-04-26: wealth USD 904.24 versus peak USD 1001.72. Maximum drawdown is 9.731%. A positive endpoint gain does not erase this earlier decline.

Discuss

1. Why would starting the running peak at the first post-loss balance hide an initial loss?
2. Does recovering half the lost dollars imply drawdown is zero? Identify the wealth level that must be reached.

Sources: [5]. See the references page.

ISSUE 10

How long does a drawdown last?

A drawdown's depth is a percentage. Its duration counts observations below a previous peak. Recovery is the first later date that reaches that peak again.

Count consecutive trading observations with D below $-1e-12$; this numerical tolerance avoids treating rounding as a new loss. A recovery date resets the count. Report the recovery of the first deepest trough separately from the longest underwater sequence: they need not be the same episode.

Before running: Does the deepest trough date necessarily identify the longest underwater duration?

```
duration = []
count = 0
for value in drawdown:
    count = count + 1 if value < -1e-12 else 0
    duration.append(count)
duration = pd.Series(duration, index=drawdown.index)
trough = drawdown.idxmin()
recoveries = wealth.loc[trough:]
recoveries = recoveries[recoveries >= peak.loc[trough] - 1e-9]
recovered_on = recoveries.index[0] if len(recoveries) else "Not in sample"
pd.DataFrame({"Deepest trough": [trough],
              "Recovery": [recovered_on], "Longest below peak (days)": [duration.max()]})
```

Deepest trough	Recovery	Longest below peak (days)
2024-04-26	2024-05-15	78

The deepest trough is 2024-04-26; recovery to its previous peak is 2024-05-15. The longest below-peak sequence is 78 trading observations. Calendar days would count weekends and holidays differently.

A sample that ends below its previous peak has no observed recovery yet. Do not substitute the sample end as a recovery date. Reaching an older peak is also different from recovering the entire initial investment if those reference amounts differ.

Discuss

1. Monday recovers the previous peak after a Friday trough. What is Monday's below-peak count? Why is elapsed calendar time a different quantity?

2. If June 28 remains underwater, what can be reported about recovery without inventing a future date?

Sources: [5]. See the references page.

ISSUE 11

What must match in a benchmark comparison?

Before comparing methods, align dates, price convention, starting cash and terminal treatment. Different exposure and turnover may remain: they are part of the strategies being compared.

Use the same AAPL dates and USD 1,000 initial capital. The fixed rule pays 4 bp on its 30 exposure changes. Buy-and-hold pays once at the January 2 entry, with no final sale. Cash earns zero. The gross rule is shown only to isolate the cost deduction.

Before running: Does a larger ending balance automatically imply a smaller maximum drawdown?

```
gross_wealth = 1000 * (1 + gross).cumprod()
buy_hold = 1000 * (1 - 0.0004) * close / close.iloc[0]
cash = pd.Series(1000.0, index=close.index)
paths = {"Rule, gross": gross_wealth, "Rule, net": wealth,
         "Buy-and-hold, net": buy_hold, "Cash": cash}
comparison = []
for name, path in paths.items():
    peak_path = path.cummax().clip(lower=1000)
    depth = -(path / peak_path - 1).min()
    comparison.append([name, path.iloc[-1], 100 * depth])
pd.DataFrame(comparison, columns=["Strategy", "Final wealth (USD)",
                                "Maximum drawdown (%)"]).round(3)
```

Strategy	Final wealth (USD)	Maximum drawdown (%)
Rule, gross	1068.799	9.077
Rule, net	1056.074	9.731
Buy-and-hold, net	1134.108	15.463
Cash	1000.0	-0.0

The net rule ends at USD 1056.07, with maximum drawdown 9.731%. Buy-and-hold ends at USD 1134.11, with drawdown 15.463%. These quantities describe different consequences; they are not an automatic overall ranking.

The buy-and-hold starting charge is included in wealth and drawdown. Its later daily returns alone omit that initial charge. Do not claim an all-cost daily comparison without explicitly allocating or separately reporting the boundary cost.

Discuss

1. Would omitting January-March only from buy-and-hold leave this comparison interpretable as a method difference? State the date correction.
2. Which two rows isolate the 4 bp cost assumption, and which comparison also changes exposure and turnover?

Sources: [1], [5]. See the references page.

ISSUE 12

Does the result depend on the chosen period?

A single endpoint can hide changes across the evaluation period. Compare stated subperiods without changing the strategy or restarting its positions at the split.

Split by return-ending date into January-March and April-June. These are calendar partitions, not independently validated market-regime labels. Each row uses the already-running account, including its dated costs; no extra entry or exit is imposed.

Before running: Could one subperiod lose money while the full period ends above its initial balance?

```
periods = {"Jan-Mar": r.loc[:"2024-03-31"],
           "Apr-Jun": r.loc["2024-04-01":]}
period_summary = []
for name, sample in periods.items():
    period_summary.append([name, len(sample),
                          100 * ((1 + sample).prod() - 1), 100 * sample.std(ddof=1)])
pd.DataFrame(period_summary, columns=["Period", "Days", "Net return (%)",
                                     "Daily volatility (%)"]).round(3)
```

Period	Days	Net return (%)	Daily volatility (%)
Jan-Mar	60	-6.48	0.759
Apr-Jun	63	12.925	1.629

January-March returns -6.480% net; April-June returns 12.925%. Their counts are 60 and 63 intervals. The total uses their compounded factors, not their arithmetic sum.

These observations describe sensitivity to the stated dates. Choosing the split after locating the worst loss can exaggerate a narrative. A claim about a market regime needs a separate, defined classification and broader evidence.

Discuss

1. If the April account is treated as a fresh purchase, which positions and costs would change relative to this table?
2. A report omits the losing subperiod and calls the remaining result typical. Which full-period and selection information should accompany that claim?

Sources: [5], [7]. See the references page.

ISSUE 13

Why might adjacent returns not be independent?

If nearby returns are dependent, resampling isolated days can remove information about their order. A sample correlation is one diagnostic, not proof of independence or a stable process.

For two uncertain returns R_1 and R_2 , variance of their sum includes twice their covariance. Below, express returns in percentage points, so variances and covariance are in pp^2 . The observed pairs are adjacent net returns, not fabricated prices.

$$\text{Var}(R_1 + R_2) = \text{Var}(R_1) + \text{Var}(R_2) + 2\text{Cov}(R_1, R_2)$$

$$3.05523 \approx 1.66601 + 1.66601 + 2(-0.13840) \text{ pp}^2$$

Before running: Does the zero-covariance calculation necessarily equal the variance of the observed paired sums?

```
previous = 100 * r.iloc[:-1].to_numpy()
following = 100 * r.iloc[1:].to_numpy()
lag_one = np.corrcoef(previous, following)[0, 1]
pair_variance = np.var(previous + following, ddof=1)
zero_covariance = np.var(previous, ddof=1) + np.var(following, ddof=1)
pd.DataFrame({"Lag-1 correlation": [lag_one],
              "Sum variance (pp²)": [pair_variance],
              "Zero-covariance (pp²)": [zero_covariance]}).round(5)
```

Lag-1 correlation	Sum variance (pp²)	Zero-covariance (pp²)
-0.08307	3.05523	3.33202

The lag-1 correlation is -0.08307. The paired-sum variance is 3.05523 pp^2 rather than 3.33202 pp^2 . This sample identity explains the omitted covariance term; it does not establish population dependence from one estimate.

Discuss

1. Would a near-zero lag-1 correlation prove independence at all lags or absence of volatility clustering? State why not.
2. What neighboring information would independent-day resampling discard that contiguous blocks partly retain?

Sources: [2], [3], [6]. See the references page.

ISSUE 14

What does a block-bootstrap interval describe?

A circular block bootstrap resamples short contiguous blocks of the observed net returns. It illustrates sampling uncertainty while keeping the fitted strategy and cost model fixed.

Use block length 5, 2,000 repetitions and seed 77. Random starts select observed returns; blocks wrap from the sample end to its beginning, then are truncated to 123 observations. That wrap is a resampling device, not a real market transition. Recalculate the zero-reference, annualized Sharpe ratio for each sample.

Before running: Predict whether the percentile interval will exclude zero. Does a positive point estimate alone establish that?

```
def bootstrap_sharpe(length):
    rng = np.random.default_rng(77)
    values = r.to_numpy()
    estimates = []
    for _ in range(2000):
        starts = rng.integers(0, len(values), int(np.ceil(len(values) / length)))
        indices = (starts[:, None] + np.arange(length)) % len(values)
        sample = values[indices.ravel()[:len(values)]]
        sd = sample.std(ddof=1)
        estimates.append(sample.mean() / sd * np.sqrt(252) if sd > 0 else np.nan)
    return np.array(estimates)
bootstrap = bootstrap_sharpe(5)
finite = bootstrap[np.isfinite(bootstrap)]
interval = np.quantile(finite, [0.025, 0.5, 0.975])
pd.DataFrame({"Block (days)": [5], "2.5%": [interval[0]],
              "Median": [interval[1]], "97.5%": [interval[2]]}).round(3)
```

Block (days)	2.5%	Median	97.5%
5	-2.423	0.629	2.852

The bootstrap Sharpe quantiles are -2.423, 0.629 and 2.852. These are ratio values, not percentage returns; 0 undefined draws were excluded. The interval includes zero under this resampling setting.

Interpretation requires sufficiently stable dependence for block resampling. It does not refit models, replay orders, account for selecting this rule, guarantee nominal coverage in this short sample, or predict future returns.

Discuss

1. Why does resampling these net returns not reproduce a new order-and-cost simulation?
2. What could change the interval besides collecting more dates? Name a resampling setting and an assumption, not a new trading rule.

Sources: [3], [6], [8], [9]. See the references page.

ISSUE 15

How much does the block length matter?

Block length changes how much neighboring order is retained. A sensitivity analysis shows whether the interval depends strongly on that choice; it does not identify a universally correct length.

Keep the same net returns, zero reference, 2,000 repetitions, seed 77, Sharpe calculation and quantiles. Change only block length among 1, 5, 10 and 20. Length 1 resamples individual days independently. Longer blocks preserve more within-block order but produce fewer blocks per resample.

Before running: Must the longest block produce the widest interval in this particular sample?

```
block_results = []
for length in [1, 5, 10, 20]:
    estimates = bootstrap_sharpe(length)
    valid_estimates = estimates[np.isfinite(estimates)]
    lower, upper = np.quantile(valid_estimates, [0.025, 0.975])
    block_results.append([length, lower, upper, len(estimates) - len(valid_estimates)])
block_results = pd.DataFrame(block_results,
                             columns=["Block (days)", "2.5%", "97.5%", "Undefined draws"])
block_results.round(3)
```

Block (days)	2.5%	97.5%	Undefined draws
1	-2.444	3.01	0
5	-2.423	2.852	0
10	-2.326	2.797	0
20	-2.259	2.735	0

Length 1 gives [-2.444, 3.010], while length 20 gives [-2.259, 2.735]. Compare the displayed endpoints, not an assumed monotone widening rule. Resetting the seed keeps the computation reproducible; changed dimensions still produce different block samples.

Selecting whichever length yields a favorable interval would be another data-dependent choice. Report the sensitivity rather than hiding it. Short history and nonstationarity remain limitations at every displayed length.

Discuss

1. If only one of these intervals excludes zero, what conclusion can be defended without selecting the favorable setting after inspection?
2. Why would changing the strategy and block length together prevent attributing the interval change to block length alone?

Sources: [3], [6], [8], [9]. See the references page.

ISSUE 16

What changes when we select the best of several rules?

Selecting the largest return among tried rules makes the reported value depend on the search. Reporting only the winner hides the alternatives that had the same opportunity to benefit from favorable sample variation.

For this separate demonstration, try positive 5-, 10-, 20- and 40-observation price-return signals. Keep AAPL data, next-Close execution and 4 bp costs fixed. Select using return-ending dates March 4-April 30, after all lookbacks are available. The main 5-observation example above remains unchanged.

Before running: Must the selected return be at least the median of the four displayed selection returns?

```

candidates = {}
for lookback in [5, 10, 20, 40]:
    feature = close / close.shift(lookback) - 1
    s = feature.gt(0).astype(float)
    p_hold = s.shift(2).fillna(0)
    a = s.shift(1).fillna(0)
    a.iloc[-1] = p_hold.iloc[-1]
    candidates[lookback] = p_hold * asset_return.fillna(0) - 0.0004 * (a - p_hold).abs()
candidates = pd.DataFrame(candidates)
selection = candidates.loc["2024-03-04":"2024-04-30"]
selection_gain = (1 + selection).prod() - 1
chosen = selection_gain.idxmax()
pd.DataFrame({"Lookback (days)": selection_gain.index,
             "Selection gain (%)": (100 * selection_gain).round(3).values})

```

Lookback (days)	Selection gain (%)
5	-2.147
10	-11.521
20	-4.345
40	0.0

The 40-observation rule is selected at 0.000%, versus a median of -3.246%. This comparison alone cannot estimate how much of the winner's result reflects chance or compute a probability of overfitting.

Trying many features, periods or cost assumptions also expands the search. Formal significance testing after such a search needs to address multiple testing; a test that treats the reported winner as the only candidate omits the selection step.

Discuss

1. Why does observing the maximum exceed the median not establish that the selected rule has a genuine predictive advantage?
2. If 100 additional rules were tried but not shown, what search information would be missing from the report?

Sources: [7]. See the references page.

ISSUE 17

What happens to the selected rule in the later period?

Evaluate the already selected rule in a later period without choosing again. A later improvement or decline is evidence about these dates, not a proof that every selected rule succeeds or fails.

The selection period ends April 30; evaluation return-ending dates are May 1-June 28. We compare the same already-running candidate paths. This is not a newly deployed portfolio switching into the winner on April 30: that would require its own entry timing and cost.

Before running: Does selecting the highest March-April result logically guarantee the highest May-June result?

```
evaluation = candidates.loc["2024-05-01":"2024-06-28"]
evaluation_gain = (1 + evaluation).prod() - 1
pd.DataFrame({"Lookback (days)": selection_gain.index,
             "Chosen earlier?": selection_gain.index == chosen,
             "Selection (%)": (100 * selection_gain).round(3).values,
             "Later (%)": (100 * evaluation_gain).round(3).values})
```

Lookback (days)	Chosen earlier?	Selection (%)	Later (%)
5	False	-2.147	16.085
10	False	-11.521	23.654
20	False	-4.345	16.583
40	True	0.0	14.808

The earlier choice remains 40 observations. Its later net gain is 14.808%, compared with 0.000% during selection. The periods differ in length and market conditions, so these totals are not equal-duration estimates.

The later results may be inspected to evaluate the frozen choice, not to rename the later winner as the original choice. Because this entire demonstration is retrospective, it is not a preregistered or untouched live test.

Discuss

1. If the instructor switches to the highest Later row, why has this period become selection data too?
2. How would a real account starting only after April 30 differ from these already-running paths? Identify the first available execution and initial trade cost.

Sources: [7]. See the references page.

ISSUE 18

Which conclusion is supported by the complete evidence?

A defensible conclusion names the strategy, dates, costs, metric and limitations. A successful code run or a favorable ratio does not replace these conditions.

Return to the unchanged five-observation rule—not the winner selected in Issue 16. Check the daily-return index, cost identity and wealth reconstruction before interpreting the summary. No new weights or trading decisions are introduced.

Before running: Should these values change merely because another candidate was selected in a separate demonstration?

```
assert r.index.equals(asset_return.iloc[1:].index)
assert np.isfinite(r).all() and (r > -1).all()
assert np.allclose(net, gross - 0.0004 * turnover)
assert np.isclose(1000 * (1 + r).prod(), wealth.iloc[-1])
pd.DataFrame({"Daily intervals": [len(r)],
              "Net gain (%)": [100 * (wealth.iloc[-1] / 1000 - 1)],
              "Max drawdown (%)": [100 * max_drawdown],
              "Annualized Sharpe": [sharpe_annual]}).round(3)
```

Daily intervals	Net gain (%)	Max drawdown (%)	Annualized Sharpe
123	5.607	9.731	0.647

The fixed rule gained 5.607% net with 9.731% maximum drawdown, using 123 daily intervals and 30 exposure changes. Its 5-day-block Sharpe interval is [-2.423, 2.852], conditional on the stated resampling assumptions.

The evidence describes one AAPL period and a hypothetical 4 bp cost model. It does not establish actual execution quality, outperformance across markets, selection-adjusted significance or future profitability. Week 8 may later compare weighting methods only after preserving these data, accounting and evaluation conditions.

Discuss

1. Write a two-sentence conclusion using the displayed gain, drawdown, period and cost assumption. Include one limitation that prevents a future-profit claim.
2. Before extending this result to another market or weighting method, which data, execution, cost and evaluation assumptions must be checked again?

Sources: [1], [2], [3], [5], [6], [7]. See the references page.

REFERENCES AND REPRODUCTION

Sources and limits

- [1] `yfinance`: historical price fields
- [2] Sharpe (1994): The Sharpe Ratio
- [3] Lo (2002): The Statistics of Sharpe Ratios
- [4] Kidd (2012), CFA Institute: The Sortino Ratio
- [5] GIPS Standards Handbook: returns and ex post risk
- [6] Politis and Romano: circular block resampling
- [7] White (2000): A Reality Check for Data Snooping
- [8] `NumPy`: quantile calculation
- [9] `arch`: fixed-length circular block resampling

Annualized calculations are teaching scale conversions, not a realized full-year record, GIPS-compliant presentation or forecast. Resampling holds the already computed strategy fixed and does not cover strategy-selection uncertainty or a new market regime. All results retain Week 6's hypothetical 4 bp costs and approximate account model.

Executed with Python 3.12.9, `NumPy` 2.2.6, `pandas` 2.2.3, `matplotlib` 3.10.8, `yfinance` 1.2.0. Run displayed code in order in a fresh Python session. Data requests need internet access; historical provider responses may change.

Selected cases illustrate the stated calculations, not investment recommendations or future-performance guarantees. Full snapshots and instructor checks are not included in the public PDF.